Text Analysis: Turning Words into Numbers

MKT 566 · Fall 2026

Davide Proserpio
USC Marshall School of Business

Before We Start

Housekeeping

- **Homework 4** is released **today** (due **Nov. 24**, on Brightspace)
- Today: **text analysis**, how words become numbers
- Thursday: **guest speaker**, Andrea Ciufo, Data Scientist, Marketing Effectiveness at dentsu

A promise from the classifiers week

The neural network slide in the classifiers deck said: in marketing, the fancy models shine on **text and images**. Today we make good on the text half.

- Reviews, tweets, chat transcripts, survey comments: most of what customers tell you arrives as **text**, not as a spreadsheet
- Every model we have used this semester (regression, logit, trees, forests) eats **rows and columns of numbers**
- So the whole game today is one move: **turn text into numbers without losing what it says**. Once that is done, the prediction step is machinery you already know, and **Homework 4 is where you run it yourselves**

Why marketers care about text

- Text reveals **what customers think** (perception), **what happened to them** (experience), and **how they feel about it** (emotion), in their own words
- What marketing teams do with it: track **brand health** across millions of posts, find the **product flaw** behind a wave of one-star reviews, route angry support tickets first, detect **fake reviews**
- The scale is the point: no team can read a million reviews, but a model can score all of them in minutes

> This is not a new kind of analytics. It is the same pipeline as always, with one extra step at the front: **text in, numbers out, then the usual models.**

From Text to Numbers

The pipeline, start to finish

Step	What it does	Example
1. Clean	lowercase, strip punctuation and HTML	“Great movie!” becomes “great movie”
2. Tokenize	split the text into words	“great movie” becomes [“great”, “movie”]
3. Drop stopwords	remove glue words that carry no meaning	“the”, “a”, “and” go away
4. Turn into numbers	one row per document, numbers in the columns	counts (bag-of-words), weighted counts (TF-IDF), **or** an average of word vectors (embeddings)
5. Model	feed the numbers to a model	the logit from week 5; you run it in Homework 4

Steps 1 to 3 prepare the text. Step 4 is **a choice between methods**: plain counts (**bag-of-words**), weighted counts (**TF-IDF**), or **embeddings**. Step 5 is **not new**: once text is a matrix of numbers, everything from weeks 4 to 9 applies unchanged.

Our running example: 5,000 movie reviews

Real data, loaded live: the `movie_review` dataset that ships with the `text2vec` R package. 5,000 IMDB reviews, each labeled **positive or negative** (2,517 positive, 2,483 negative).

Two real examples, shortened:

> "the classic war of the worlds" by timothy hines is a very entertaining film that obviously goes to great effort and lengths to faithfully recreate h. g. wells' classic book. mr. hines succeeds in doing so. i, and those … (**positive**)

> this movie could have been very good, but comes up way short. cheesy special effects and so-so acting. i could have looked past that if the story wasn't so lousy. if there was more of a background story, it would have be… (**negative**)

After cleaning, these 5,000 reviews contain about **1.2 million words** drawn from **41,891 distinct terms**. Nobody reads that; the pipeline turns it into a table.

Steps 1 and 2: clean, then tokenize

One sentence, worked through:

Stage	Result
Raw	`Great movie!!   A must-see... 10/10`
Lowercase	`great movie!!   a must-see... 10/10`
Strip HTML and punctuation	`great movie a must see 10 10`
Tokenize (split on spaces)	`great` · `movie` · `a` · `must` · `see` · `10` · `10`

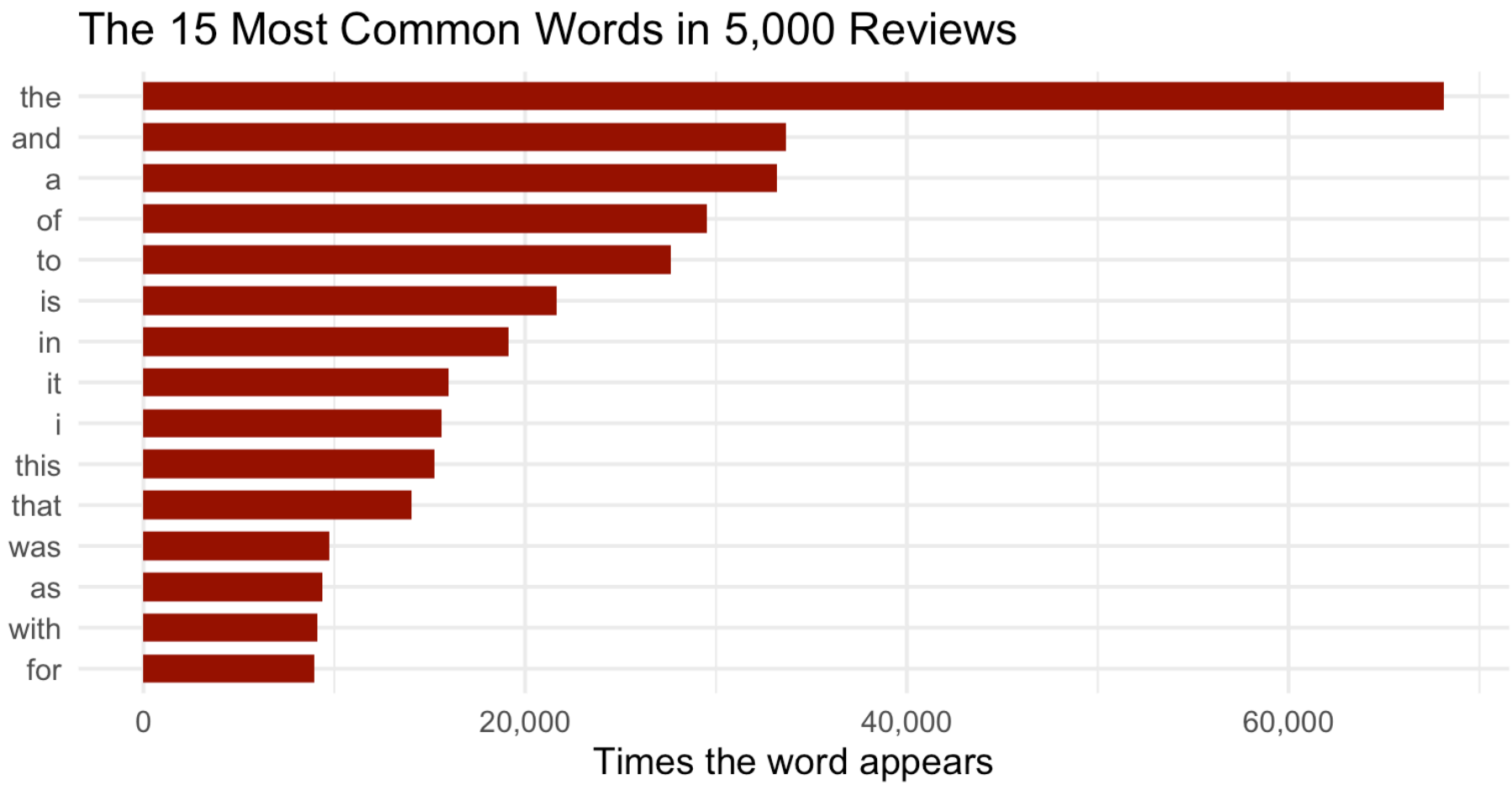
- Each piece is a **token**. From here on, the model never sees sentences, only tokens and their counts
- Real text is dirty: these reviews contain leftover HTML like `<br />`, which would otherwise become a nonsense token. **Look at your tokens before you model them**

Step 2 continued, n-grams: when one word is not enough

Splitting into single words destroys word order, and sometimes the order is the meaning:

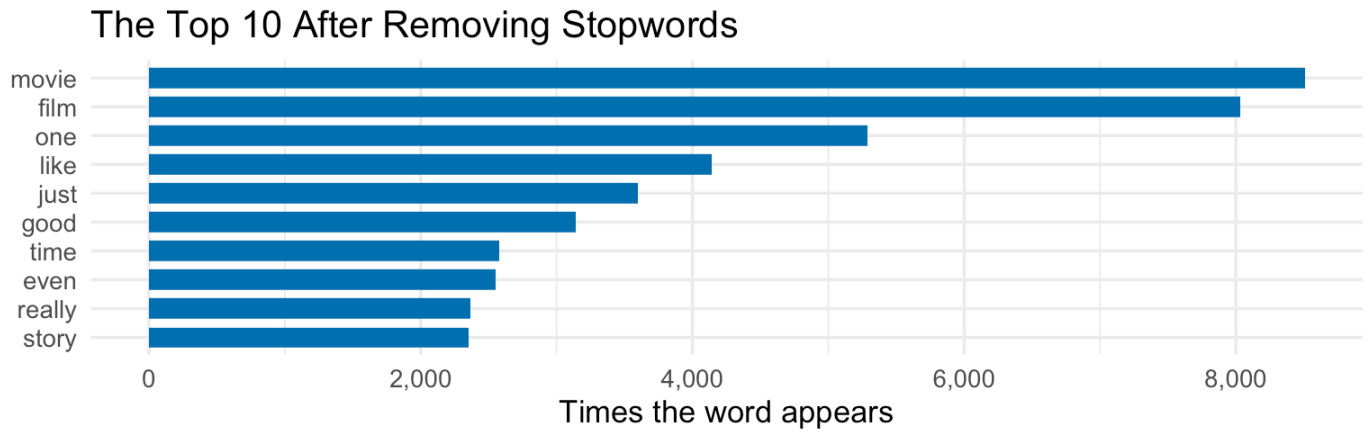
- “good” and “**not** good” say opposite things, but as single tokens they both contribute the word `good`
- The fix: also count **pairs of consecutive words**, called **bigrams**: `not_good` becomes its own token, distinct from `good`
- Single words are **unigrams**; you can go to trigrams and beyond, but the number of distinct terms explodes, so most marketing work stops at bigrams

Step 3, the problem: the most common words



Every word in all 5,000 reviews, counted. What do you notice about the words at the top, and how useful would they be for telling a good review from a bad one?

Step 3, the fix: stopwords get removed



- The raw top 15 was all **the, a, and, of**: grammar glue. These words appear in every document, so they cannot separate anything from anything
- Words like these are called **stopwords**, and step 3 simply deletes them from a standard list (the list is a choice: in some problems “not” matters, so check before deleting it)
- After removal, the top words are **movie, film, story, time**: now the counts start to mean something

The three methods we will learn

Three ways to turn text into numbers, in increasing order of sophistication:

	Method	In one line
1	**Bag-of-words**	count how often each word appears in each document
2	**TF-IDF**	re-weight the counts so distinctive words stand out
3	**Embeddings**	give every word a position on a map of meaning

- Methods 1 and 2 we do **by hand**, on a corpus of three tiny reviews, and they are what you will use in **Homework 4**
- Method 3 you only need to **recognize**: it is the idea that modern language models build on
- Whichever method you choose, step 5 of the pipeline is the same: feed the matrix to the **models you already know**

Step 4 (method 1), bag-of-words: count the words

Take a tiny, made-up corpus of three phone reviews, and **count every word in every document**:

- A: “great phone great camera”
- B: “terrible battery terrible screen”
- C: “great battery”

doc	battery	camera	great	phone	screen	terrible
A	0	1	2	1	0	0
B	1	0	0	0	1	2
C	1	0	1	0	0	0

- This table is the **bag-of-words matrix**: one row per document, one column per word, counts in the cells. Review A **is** the vector (0, 1, 2, 1, 0, 0)
- “Bag” because order is thrown away: “great phone” and “phone great” give the same row. That is the price of turning text into a table

Bag-of-words at real scale

On the real corpus, the same table is 5,000 rows by **41,891 columns**, one column for every distinct term.

- Almost every cell is **zero**: a 200-word review touches 200 of the 41,891 columns. This is called a **sparse matrix**, and R stores only the non-zero cells
- Simple and interpretable: every column is a word you can read
- Two problems remain: common words get big counts **just for being common**, and the table is now **wider than it is tall**. A regression needs more reviews than columns, so with more columns than reviews it simply breaks
- The first problem we fix on the next slide, with TF-IDF. The second you will fix in **Homework 4** the simple way: **keep only the 1,000 most common words**. A word that shows up in just a few reviews cannot teach the model much, so dropping it costs almost nothing

Step 4 (method 2), TF-IDF: weight the counts

Method 1 counts; method 2 **re-weights those counts** so that the words that identify a document stand out. A word matters for a document when it is **common in that document but rare everywhere else**. TF-IDF scores exactly that, in two parts:

- **TF (term frequency)**: how much of document d is word t? $\mathrm{TF}(t, d) = \frac{\text{count of } t \text{ in } d}{\text{words in } d}$
- **IDF (inverse document frequency)**: how rare is the word across documents? $\mathrm{IDF}(t) = \log \frac{N}{\text{docs containing } t}$, with N the number of documents. A word in **every** document gets $\log(1) = 0$: no weight at all
- **TF-IDF** $= \mathrm{TF} \times \mathrm{IDF}$: big only when both parts are big
- The score is **never negative**: 0 means "this word is everywhere". And there is no fixed maximum (the ceiling grows with the number of documents), so compare TF-IDF scores **within** a corpus, never across corpora

TF-IDF by hand: “great” in document A

Document A is “great phone great camera”: **4 words**. The corpus has $N = 3$ documents. One word at a time, starting with “great”:

Step	Question	Arithmetic	Result
1. Count	how often is “great” in A?	twice	2
2. TF	what share of A is “great”?	2 / 4	0.50
3. DF	how many documents contain “great”?	A and C	2 of 3
4. IDF	how rare is that?	log(3 / 2)	0.405
5. TF-IDF	TF times IDF	0.50 × 0.405	**0.203**

- “great” fills **half** of document A, a big TF. But 2 of the 3 documents contain it, so its IDF is small: the corpus treats it as **ordinary**
- log here is the **natural log**: `log()` in R, *ln* on a calculator

Same recipe for “phone”

Step	Question	Arithmetic	Result
1. Count	how often is “phone” in A?	once	1
2. TF	what share of A is “phone”?	1 / 4	0.25
3. DF	how many documents contain “phone”?	only A	1 of 3
4. IDF	how rare is that?	log(3 / 1)	1.099
5. TF-IDF	TF times IDF	0.25 × 1.099	**0.275**

- “phone” appears **half as often** as “great”, so its TF is half as big
- But **no other document has it**, so its IDF is almost three times bigger
- Rarity wins: **0.275 beats 0.203.** “phone” identifies document A; “great” does not

All of document A, scored

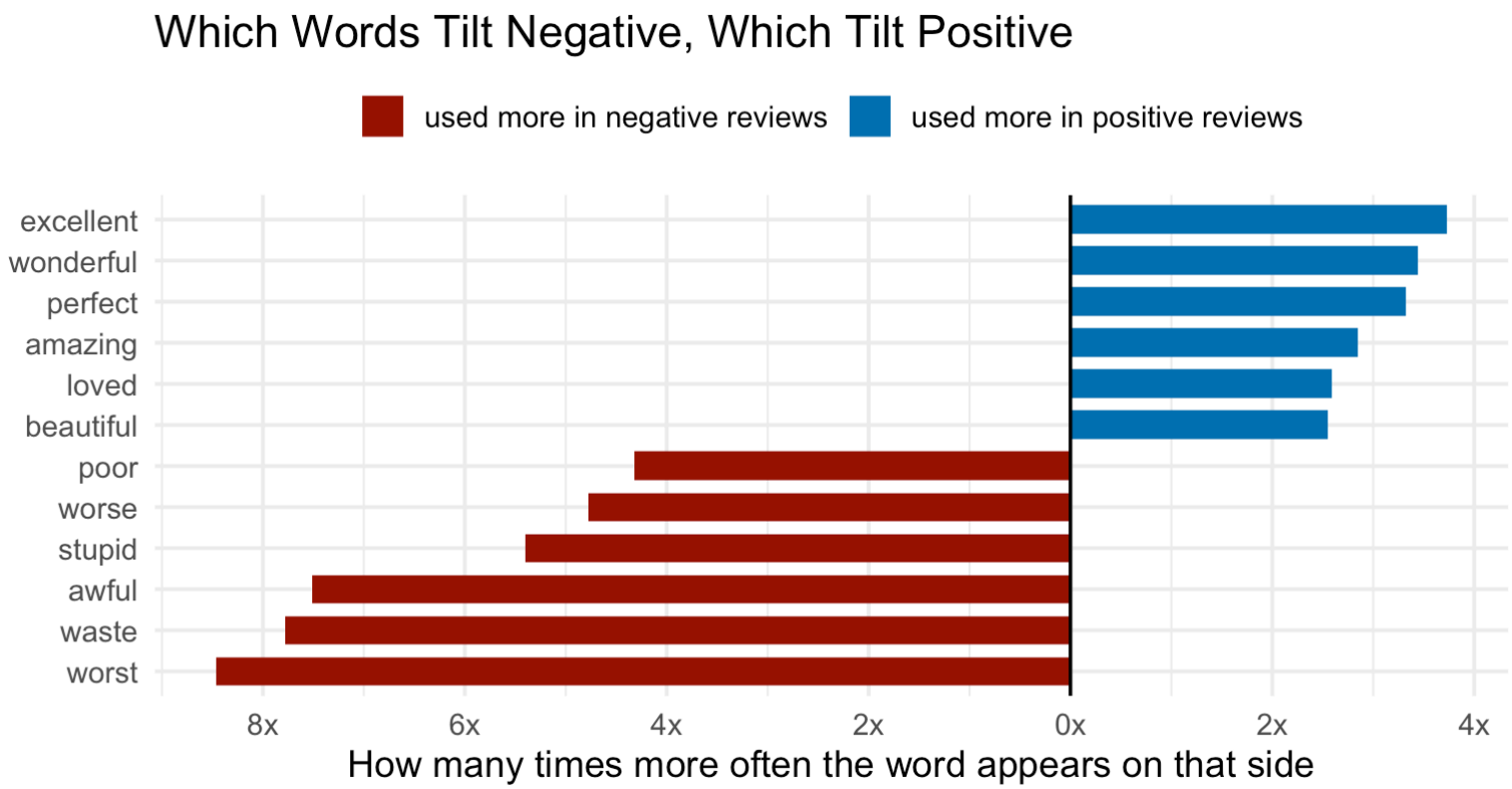
The same five steps for every word of A:

word	count in A	TF = count/4	docs with word	IDF = log(3/df)	TF-IDF
camera	1	0.25	1	1.10	0.275
phone	1	0.25	1	1.10	0.275
great	2	0.50	2	0.41	0.203

- “camera” scores like “phone”: once in A, and no other document has it
- That is the whole idea: **“great” is everywhere; “phone” and “camera” tell you which review this is.** The model should listen to them more

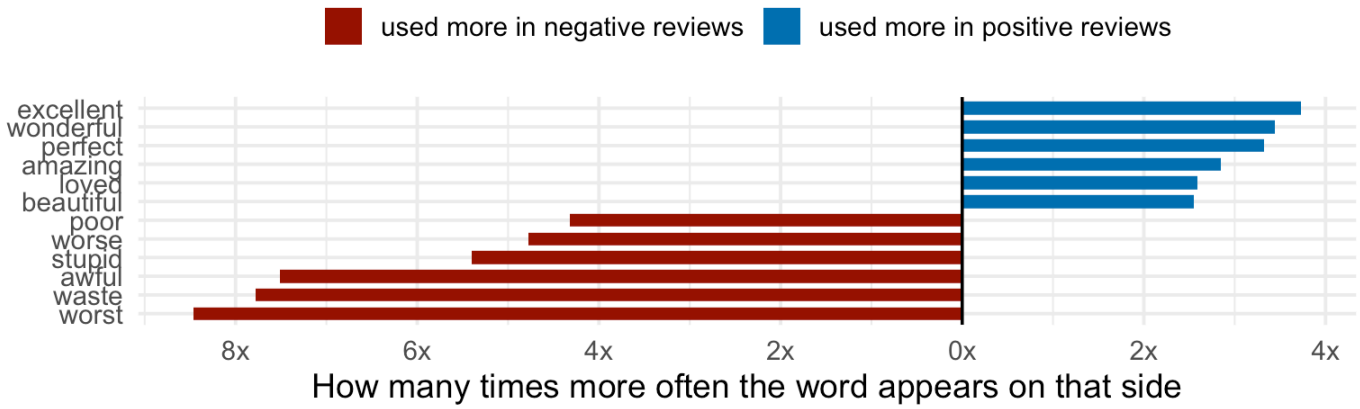
The payoff: which words separate good from bad

We now have text as numbers. The question that justifies all the counting: do those numbers **carry signal**? Our 5,000 reviews come **labeled** positive or negative, so we can simply compare which words each side uses more:



For every word, we counted how often positive reviews use it and how often negative reviews use it (per 10,000 words), then took the ratio. “worst” at 8x on the left means negative reviews use it 8 times more often than positive ones. What do you see?

Reading the chart



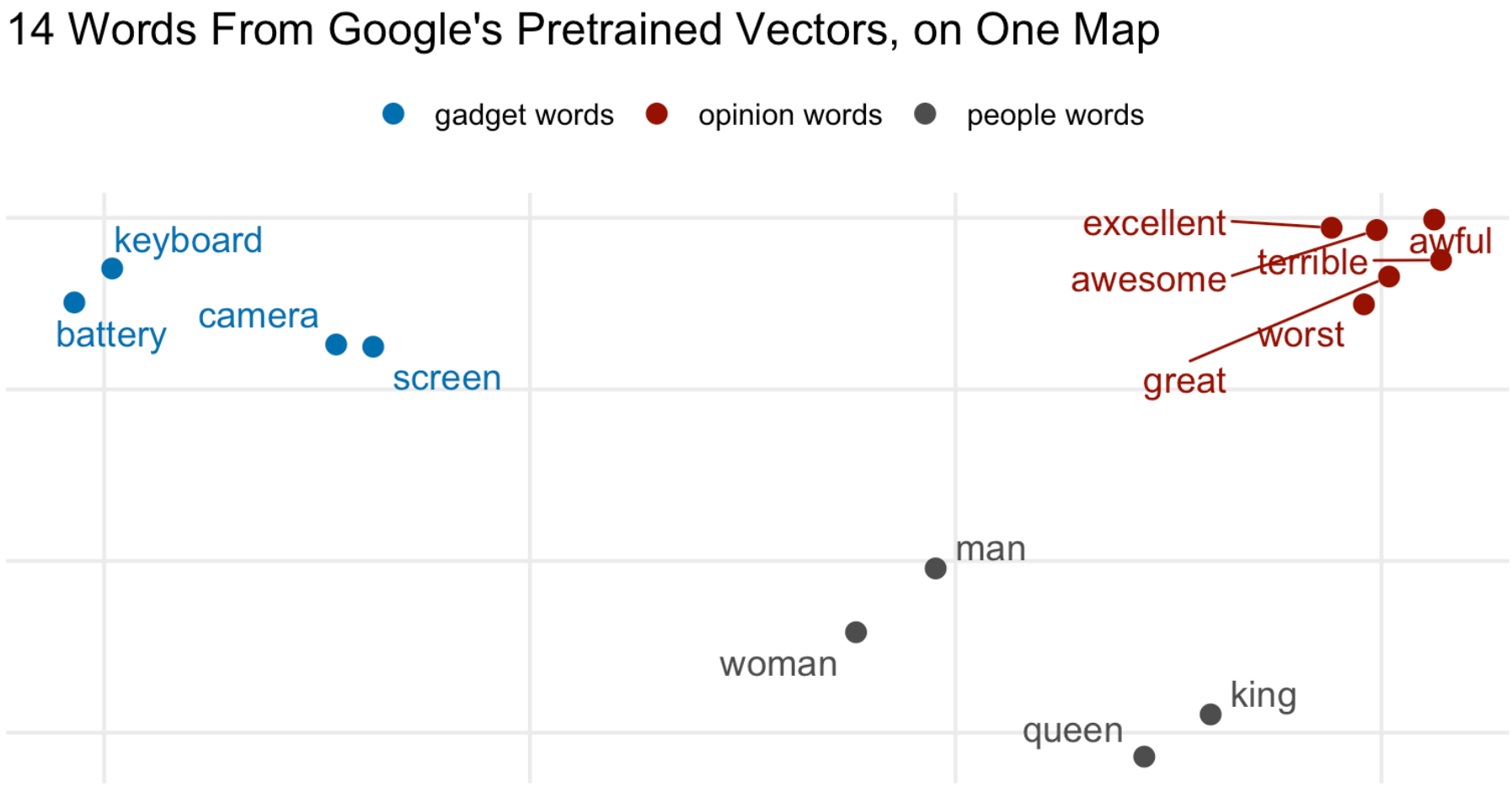
- Negative reviews lean on **worst, waste, awful, stupid**: direct complaint words. Positive reviews lean on **excellent, wonderful, perfect, amazing**
- Nobody told the data which words are “good”: the labels plus simple counting surfaced them. This is exactly the signal your Homework 4 model will learn from
- The two sides are not symmetric: the sharpest negative word tilts about **8x**, the sharpest positive one only about **4x**. Complaints use extreme words; praise words also show up inside negative reviews (“the trailer was great, the movie was not”). **No single word decides a review**, which is why the model adds up thousands of small signals

Method 3, word embeddings, in plain words

Methods 1 and 2 share a blind spot: both treat “great” and “awesome” as unrelated as “great” and “battery”: **different columns, nothing shared**. TF-IDF changes the numbers in the columns, not the columns themselves.

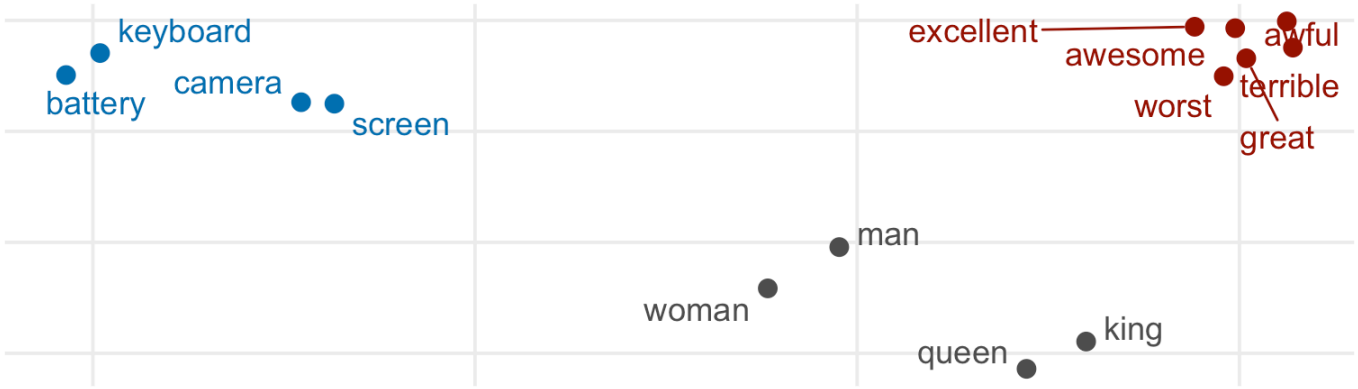
- **Embeddings** fix that by giving every word a list of, say, 300 numbers (a **vector**), learned so that words used in similar contexts get **similar vectors**: awesome ends up close to great, far from battery
- They are learned from huge text collections by models like **word2vec** (guess the word from its neighbors), and you download them **pretrained** rather than training your own
- The trade-off by now is familiar: embeddings capture **meaning** but the 300 numbers are unreadable; counts are readable but treat every word as an island

The map of meaning, drawn



Each word's 300 numbers, squashed onto one flat map (the axes have no units; only distances matter). Nobody told the computer what these words mean. What do you see?

Reading the map



- **Similar words sit together**, in groups nobody labeled: gadget words, people words, opinion words. That is what “similar contexts get similar vectors” looks like
- Look closely at the opinion cluster: **“awful” sits right next to “awesome”**. Embeddings know these words are used in the same kinds of sentences, not whether the sentence was happy. For good vs bad you still need labels, which is where the classifier comes in

The three methods, side by side

Method	Each document becomes	Strength	Weakness
Bag-of-words	counts over the vocabulary	simple, readable	common words dominate; synonyms unrelated
TF-IDF	weighted counts	highlights distinctive words; the workhorse	still ignores meaning and order
Embeddings	an average of word vectors	captures meaning (great is close to awesome)	unreadable numbers; needs a pretrained model

Default advice for marketing problems: **start with TF-IDF**. It is fast, transparent, and with a linear model it is a strong baseline. Try embeddings only if TF-IDF is not good enough, and keep them only if they **predict better on your test set**. Homework 4 tests what the text is worth on 20,000 real reviews: the same models with and without the word columns.

Measuring Opinion: Sentiment

Sentiment: the first thing marketers ask of text

- **Sentiment** is the polarity of an opinion: positive, negative, or neutral. “Is this review happy or angry?”
- Where it earns money: track **customer satisfaction** without surveys, watch a **campaign** land in real time, spot a **brand crisis** while it is still small, prioritize the angriest **support tickets**
- Two ways to measure it: **train a classifier** on labeled reviews, or use a ready-made **dictionary**

Way 1: train a classifier

Supervised sentiment is **week 9 on text features**. Every step is something you already know:

Step	What you do	Where you learned it
1. Label	each review marked positive or negative (ours already are)	the outcome, week 5
2. Vectorize	TF-IDF matrix: 5,000 rows, thousands of word columns	today
3. Split	train/test, fit on train only	week 9
4. Classify	a logit on the word columns	week 5, keeping only the most common words so the matrix stays manageable
5. Evaluate	accuracy, precision, recall, AUC on the test set	week 9

> Nothing here is new: it is the week 9 recipe with words as the features. The catch is the first row: you need **labels**, thousands of reviews already marked positive or negative. What if you have none? That is way 2.

Way 2: a dictionary of words

No labels? Use scores somebody else already assigned. Lexicon-based sentiment (the `sentimentr` package in R):

- Start from a **dictionary**: thousands of words pre-scored by humans (great = +0.75, terrible = -0.75, table = 0)
- Score a sentence by **averaging the scores of its words**
- The good packages also handle **valence shifters**: "**not** great" flips the sign, "**absolutely** terrible" amplifies it. This is why they work at the sentence level, not word by word

sentimentr, live on four sentences

Sentence	Sentiment score
This movie is great.	0.25
This movie is not great.	-0.22
Absolutely terrible, I want my money back.	-0.28
The plot is slow at first, but the ending is fantastic.	0.13

- Sentence 1 vs. sentence 2: the **“not”** flips a clearly positive score to a negative one. A plain word count would have scored both as positive, because both contain “great”
- Sentence 3: “absolutely” **amplifies** “terrible”, the most negative score of the four
- Sentence 4 is the honest case: mixed opinions inside one review average out to a mildly positive score. Real reviews are full of this, which is why sentence-level scoring beats word counting

Classifier or dictionary?

	Way 1: classifier	Way 2: dictionary
Needs	thousands of labeled reviews	nothing: works on day one
Knows your domain	yes: learns from your own reviews that “thin” is bad for a battery	no: one score per word, the same everywhere
Accuracy	usually higher, and you can **measure** it on your test set	lower, and without labels you cannot even measure it
Cost	collecting the labels	free

Rule of thumb: **have labels? Train the classifier. No labels? Start with the dictionary**, then hand-label a few hundred reviews to check it before you trust it at scale.

Beyond positive vs. negative

Two directions you should know exist, both one Google away:

- **Emotion detection**: dictionaries that map words to emotions rather than polarity (the NRC lexicon tags ~14,000 words with joy, anger, fear, trust…). “Which campaign made people excited, and which made them anxious?”
- **Topic modeling** (LDA): unsupervised grouping of documents by the words they share, clustering for text. “What are the five things people complain about?” It finds the themes; **you** name them, exactly like the customer segments in week 6

What text analysis will not tell you

- **Sarcasm and context**: “Great, it broke after two days” scores positive on words. Dictionaries fall for it; trained models only partly escape it
- **The domain shifts**: a dictionary built on movie reviews mis-scores banking complaints. Re-check whenever you change domains
- **Volume is not representativeness**: the people who write reviews are not your average customers (week 1’s Groupon paper all over again)
- **Correlation is not causation**, still: customers who write angry reviews often cancel soon after, so text is great for flagging **who** is at risk. But the angry words are a **symptom** of a bad experience, not the cause of leaving: both the review and the cancellation come from whatever went wrong. Fix what went wrong, not the words

Wrap-up

The R code behind every chart and table:

- **`w11-code.zip`**: the class script (the toy corpus, the frequency charts, TF-IDF by hand, the sentimentr demo)
- **`w11-1-text-analysis-class.R`**: same content, runnable line by line; the reviews load from the `text2vec` package, no download needed

Required reading: chapter 8 of *Introduction to R for Data Science*. Recommended: the HBR piece on fake reviews. Practice on your own, linked on the course page: the Harry Potter sentiment replica and the mini TF-IDF exercise.

Questions?